

Contents

Detailed Build Plan: Filesystem MCP Server Integration	1
1. Purpose in the Control Panel	1
2. Integration Points	1
3. Detailed Implementation Steps	1
4. Code Architecture	2
5. Security Considerations	2
6. Testing Strategy	2
7. Multi-Perspective Summary	2

Detailed Build Plan: Filesystem MCP Server Integration

MCP Server: Filesystem (@modelcontextprotocol/server-filesystem)

Type: stdio

Priority: High (Foundational)

Phase: Primary focus in Phase 3

1. Purpose in the Control Panel

Enable the Tauri Control Panel to give Grok Build agents secure, controlled read/write access to directories **outside** the current project (docs, assets, shared configs, templates, etc.).

2. Integration Points

- **Config & Extensions Browser** → Dedicated “Filesystem MCP” tab
- **Session Orchestrator** → Option to attach specific paths when spawning sessions
- **Permission & Sandbox Controller** → Map to sandbox profiles + path allow/deny rules
- **McpManager service** → Core CRUD + validation

3. Detailed Implementation Steps

Shared Infrastructure (Do First)

- Create `McpServerConfig` struct with fields: `name`, `transport` (stdio), `command`, `args`, `env`, `startup_timeout_sec`, `tool_timeout_sec`, `enabled`, `scope` (global/project)
- Implement `McpManager` with methods: `list()`, `add()`, `remove()`, `update()`, `doctor()`, `get_tools()`
- Add typed CLI wrapper methods in `grok_cli_wrapper`

Filesystem-Specific

1. **Add Server Flow**
 - UI: Path picker + optional read-only toggle
 - Backend: Validate path exists and is accessible
 - Generate command: `npx -y @modelcontextprotocol/server-filesystem /absolute/path`
 - Support multiple paths (comma-separated or repeated args)
2. **Security Hardening**
 - Whitelist base directories (user home, project parent, specific safe paths)
 - Prevent adding sensitive paths (`/`, `/etc`, `~/.ssh`, etc.)
 - Store as project-scoped by default when possible
3. **Session Injection**
 - When spawning a session, allow selecting which filesystem MCP servers to attach
 - Pass via MCP config merging or environment
4. **Tool Exposure**
 - Expose common tools: `read_file`, `write_file`, `list_directory`, `search_files`, etc.
 - Show in session tool list with namespace `filesystem_*`

4. Code Architecture

- New file: `grok_control_core/src/mcp/filesystem.rs`
- Extend `McpManager` with filesystem-specific validation helpers
- Add to `SpawnOptions`: `mcp_servers: Vec<String>`

5. Security Considerations

- Path traversal prevention
- Read-only mode support (via server flags if available)
- Audit log for all file operations triggered via MCP

6. Testing Strategy

- Unit tests for path validation
- Integration test: Add filesystem MCP → spawn session → verify agent can read/write approved path
- Doctor test via `grok mcp doctor`

7. Multi-Perspective Summary

- **Security:** High priority on path whitelisting
- **Usability:** Simple path picker in UI
- **Performance:** stdio servers have cold-start cost → increase `startup_timeout_sec`

Dependencies: None (foundational)

Estimated Effort: Medium (core shared + one server)